

Lecture Unit 8.4

OpenMP tasks

Introduction

- Previously we have seen how OpenMP can parallelise loops and sections
- There are other cases where we need to express parallelism which is not described by loops with a predetermined trip count or number of sections:
 - Recursive algorithms
 - Traversing a linked list
- OpenMP 3.0 introduced tasks to handle these cases

OpenMP tasks

- One thread generates tasks and other threads execute the tasks
- After the generating thread has finished generating the tasks it may carry out some of the tasks
- A **single** construct is used to ensure that only one thread generates tasks

tasks.c

```
#include <stdio.h>

int main(){

#pragma omp parallel
{
#pragma omp single
{
    printf("Generating tasks:\n");

#pragma omp task
    {
        printf("Task 1\n");
    }

#pragma omp task
    {
        printf("Task 2\n");
    }

#pragma omp task
    {
        printf("Task 3\n");
    }
} // single
} // parallel

printf("Tasks done\n");

return 0;
}
```

One thread generates tasks
(**single** construct)
and the other threads
can run tasks

tasks.c

```
> gcc -fopenmp -o tasks tasks.c
> export OMP_NUM_THREADS=2
> ./tasks
Generating tasks:
Task 1
Task 3
Task 2
Tasks done
> ./tasks
Generating tasks:
Task 1
Task 2
Task 3
Tasks done
> ./tasks
Generating tasks:
Task 1
Task 3
Task 2
Tasks done
> ./tasks
Generating tasks:
Task 1
Task 2
Task 3
Tasks done
>
```

Make sure there are
more tasks than threads

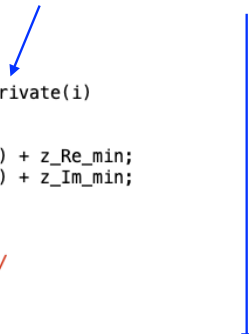
Tasks can be carried out
in any order

Mandelbrot with OpenMP tasks:

```
/* Loop over real and imaginary axes */
#pragma omp parallel
{
    #pragma omp single
    {
        for (i=0; i<nRe+1; i++){
            #pragma omp task default(shared) private(j, k, z_Re, z_Im, z0, z) firstprivate(i)
            {
                for (j=0; j<nIm+1; j++){
                    z_Re = ( (float) i) / ( (float) nRe) * (z_Re_max - z_Re_min) + z_Re_min;
                    z_Im = ( (float) j) / ( (float) nIm) * (z_Im_max - z_Im_min) + z_Im_min;
                    z0 = z_Re + z_Im*I;
                    z = z0;

                    /* Iterate up to a maximum number or bail out if mod(z) > 2 */
                    k=0;
                    while(k<maxIter){
                        nIter[i][j] = k;
                        if (cabs(z) > 2.0)
                            break;
                        z = z*z + z0;
                        k++;
                    } // while
                } // j
            } // task
        } // i
    } // single
} // parallel
```

Each task has a different value of `i` but the value needs to be initialised from outside the task



“The `firstprivate` clause declares one or more list items to be private to a task, and initialises each of them with the value that the corresponding original item has when the construct is encountered.”

<https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-5.0.pdf>

Unit summary

- In this unit we have looked at OpenMP tasks
- Tasks provide a more flexible form of parallelism than parallelising loops and sections
- A **single** construct is used so that one thread generates tasksOther threads in the parallel region carry out the tasks
- We also saw the **firstprivate** clause which initialises a private variable with the value from outside the task construct